IG.A.02 CRITERIOS DE CÁTEDRA

Criterios de programación estructurada

Fundamentos, pautas de resolución y ejemplos comparativos en Python.

Versión 1.3 · Última modificación: 09/11/2026 — 18:11 h

Este documento presenta tres criterios didácticos de Informática General (IG) para resolver ejercicios en Python: no utilizar break, escribir un único return al final de cada función y expresar las comparaciones mediante operadores lógicos explícitos. Son pautas independientes, aplicadas de acuerdo con los contenidos enseñados, que ayudan a unificar el trabajo de estudiantes y docentes. No significan que otras construcciones válidas de Python sean incorrectas en todos los contextos.

Uso de este material con asistentes de IA

Si utilizás una IA para estudiar, adjuntá esta guía de criterios, la consigna del ejercicio y **IG.A.01 — Guía para estudiar con asistentes de IA**. Compartí también los PDF de teoría y práctica pertinentes y los otros materiales de apoyo que correspondan.

INSTRUCCIÓN PARA COPIAR EN LA IA

Actuá como tutor de Informática General (UCA). Antes de responder, leé la consigna y los documentos adjuntos. Adaptá las explicaciones, los ejemplos, las correcciones y las soluciones a los criterios de la cátedra y a los contenidos enseñados. Ayudame a comprender el razonamiento y revisar mis intentos. Si solicito una solución completa, ajustala también a esas pautas. No propongas como solución principal recursos que los documentos indiquen como no admitidos, aunque sean válidos en Python. Mencioná qué criterio aplicás. Si falta información o no podés leer algún archivo, avisame; no inventes restricciones.

Para este informe, aplicá especialmente los criterios de las secciones 3, 4 y 5. La consigna específica de cada actividad también debe respetarse.

1. Fundamento pedagógico de los criterios

En IG se solicita resolver los ejercicios con las herramientas desarrolladas hasta la unidad correspondiente. En particular, se trabaja sin break, con un único return al final de cada función y con comparaciones expresadas mediante operadores lógicos explícitos.

El objetivo es favorecer el razonamiento sobre el estado de las variables, las condiciones de continuidad de los ciclos y el resultado de las funciones. Estos criterios permiten practicar y evaluar de manera homogénea los contenidos enseñados.

- Seguimiento del flujo de control: identificar qué instrucciones se ejecutan y cómo cambian las variables durante el programa.
- Construcción de condiciones: formular expresiones con and, or y not para controlar decisiones y repeticiones.
- Organización de funciones: calcular un resultado y devolverlo mediante un único return al final de la función.

2. Alcance de las pautas en una materia inicial

Informática General es una asignatura introductoria y transversal a distintas carreras de Ingeniería. Las pautas buscan construir una base común para analizar problemas, diseñar algoritmos, implementar soluciones y verificar resultados.

- **Criterio didáctico:** Las restricciones delimitan las herramientas que deben dominarse en esta etapa; no constituyen reglas universales de Python ni de la programación profesional.
- **Criterio de evaluación:** La corrección considera tanto el funcionamiento del programa como la forma en que se resolvió: se espera la aplicación correcta de los conceptos, recursos y estructuras enseñados, respetando los criterios de la cátedra y las pautas de cada actividad o evaluación. Por lo tanto, que un programa funcione no es, por sí solo, suficiente para cumplir todos los criterios de corrección.

3. Uso no admitido de `break` en Informática General

¿Qué hace en Python? La instrucción `break` finaliza inmediatamente el ciclo `for` o `while` en el que se encuentra, aunque la condición del ciclo todavía permita continuar. Es una instrucción válida de Python, pero **no forma parte de los contenidos enseñados ni admitidos en IG**.

¿Qué buscamos enseñar? Que el estudiante identifique cuándo debe continuar o terminar una repetición y exprese esa decisión en la condición del `while`, utilizando las variables necesarias. En una búsqueda, por ejemplo, puede combinarse el índice con una variable booleana que indique si se encontró el elemento.

¿Por qué importa? Este enfoque permite practicar el seguimiento de variables, la formulación de condiciones lógicas y el control explícito de las iteraciones. No significa que `break` sea incorrecto o inseguro en otros programas.

Ejemplo 1. Búsqueda de un elemento en una lista

NO ADMITIDO EN IG Enfoque con Interrupción

```
encontrado = False
for elemento in lista:
    if elemento == "objetivo":
        encontrado = True
        break  # No admitido en IG
```

FORMA ESPERADA EN IG Enfoque con Ciclo Natural y Bandera

```
encontrado = False
indice = 0
# La condición controla la finalización
while indice < len(lista) and not encontrado:
    if lista[indice] == "objetivo":
        encontrado = True
    indice += 1
```

> **Criterio de IG:** el ciclo debe finalizar por su condición de continuidad, no mediante `break`. Si el ejercicio requiere detener la búsqueda al encontrar un elemento, esa situación debe incorporarse a la condición del ciclo.

4. Uso de `return`: un único retorno al final de cada función

¿Qué hace en Python? La instrucción `return` termina la ejecución de una función y, si corresponde, devuelve un valor al código que la llamó. Python permite escribir varios `return` en una misma función, incluso dentro de estructuras `if` o ciclos.

¿Cuál es la pauta de IG? En las funciones que devuelven un resultado, se utiliza **un único `return`, al final de la función**. Antes de llegar a él, el programa debe calcular o determinar el valor que se devolverá, por ejemplo, mediante una variable `resultado`.

¿Qué buscamos enseñar? Separar el proceso de cálculo de la devolución del resultado, seguir todas las alternativas de una estructura condicional y reconocer un punto de salida común. Una función puede tener distintas variables locales y varios caminos de decisión: lo que se unifica es el retorno, no toda la lógica interna.

Relación con `break`: ambas instrucciones pueden interrumpir el recorrido habitual del código, pero actúan en ámbitos diferentes: `break` termina un ciclo; `return` termina una función. Por eso se explican y evalúan como **dos criterios distintos**.

Ejemplo 2. Validación de un monto

NO ADMITIDO EN IG Múltiples Puntos de Salida

```python
def evaluar_monto(monto):
    if monto <= 0:
        return "Invalido"  # Retorno anticipado o adicional
    if monto > 1000:
        return "Excedido"  # Retorno anticipado o adicional
    return "Aprobado"      # Tercer return de la función
```

FORMA ESPERADA EN IG Un único retorno al final de la función

```python
def evaluar_monto(monto):
    resultado = ""  # Variable que almacena el resultado

    if monto <= 0:
        resultado = "Invalido"
    elif monto > 1000:
        resultado = "Excedido"
    else:
        resultado = "Aprobado"

    return resultado  # Único return al final de la función
```

> **Criterio de IG:** no se admiten retornos anticipados ni varios `return` en una misma función. Esta pauta didáctica no implica que los retornos anticipados sean siempre una mala práctica en el desarrollo profesional.

5. Comparaciones encadenadas: uso no admitido en Informática General

¿Qué permite Python? Escribir comparaciones encadenadas, como `10 <= x <= 20`. Esta sintaxis es válida en Python.

¿Qué solicitamos en IG? Expresar cada comparación por separado y relacionarlas con el operador lógico correspondiente: `10 <= x and x <= 20`. Así el estudiante practica tanto las comparaciones como el uso explícito de `and`.

Para variables simples, las dos expresiones del ejemplo comprueban el mismo rango. La segunda es la **forma esperada en IG**; no se afirma que sea siempre superior en Python ni que todas las expresiones encadenadas sean intercambiables sin considerar su evaluación.

Ejemplo 3. Comprobación de un rango

NO ADMITIDO EN IG Encadenamiento Implícito

```
if 10 <= x <= 20:  # Válido en Python; no admitido en IG
    print("Dentro del rango")
```

FORMA ESPERADA EN IG Operadores Lógicos Explícitos

```
if 10 <= x and x <= 20:  # Comparaciones explícitas con and
    print("Dentro del rango")
```

> **Criterio de IG:** escribir explícitamente las comparaciones y combinarlas mediante operadores lógicos, según corresponda al problema.

Conclusión

Los tres criterios tienen un propósito formativo común, pero se refieren a aspectos diferentes: la finalización de ciclos (`break`), la devolución del resultado de funciones (`return`) y la expresión de condiciones lógicas (`and`).

Si bien las alternativas presentadas son válidas en Python y pueden resultar apropiadas en determinados contextos profesionales, **en Informática General se espera y se evalúa la aplicación de los criterios de programación establecidos por la cátedra**, de acuerdo con los contenidos y las herramientas enseñadas.

Por lo tanto, **no alcanza con que un programa funcione correctamente**: también debe estar desarrollado utilizando los recursos y las estructuras de programación correspondientes a la etapa de aprendizaje y respetando las pautas establecidas para cada actividad o evaluación.